Chopping binary executables

Object file recovery and Modular Decomposition

Chariton Karamitas,
CENSUS S.A. & AUTh,
Greece
huku@census-labs.com

March 16, 2026

Table of contents

1 Whoami

2 Problem statement

3 REcover

4 Modular decomposition

5 Conclusions

6 Questions

Whoami

- huku a.k.a. Chariton Karamitas
- Vulnerability researcher at CENSUS
  - Android only since 2015
- Binary diffing PhD
- Proud Phrack author

Problem statement

Problem statement

- The Android ecosystem is heavily fragmented across vendors and devices
- My daily work involved analyzing differences/similarities in various vendors' closed-source components
- Frequent exposure to binary diffing sparked my research interest in the area
- Decided to pursue a PhD

Problem statement

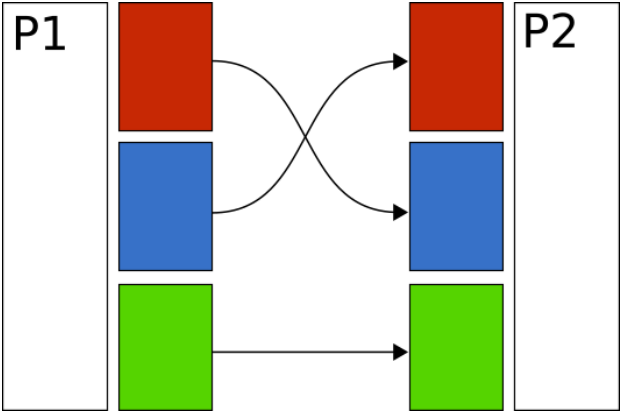
- Literature review showed a growing trend towards learning-based approaches to binary diffing
- Examples:
  - Graph Matching Networks for Learning the Similarity of Graph Structured Objects (2019)
  - DeepBinDiff: Learning Program-Wide Code Representations for Binary Diffing (2020)
- These are interesting research directions, but I'm not a huge fan of machine learning
- I was mostly interested in more graph-theoretic approaches

Problem statement

- Brainstormed for a few days
- Can we design divide-and-conquer algorithms for binary diffing?
  - Potential improvements in performance (i.e., speed)
  - Potential improvements in efficiency (i.e., function matching precision)
  - Program decomposition is an interesting research problem on its own
- Decided to pursue this direction

Problem statement

This is, more or less, what I had in mind:



Problem statement

- We examine two techniques for decomposing binary executables into smaller components:
  - REcover
    - Approximate recovery of stripped executables' object file structure
    - Decomposition into groups of "semantically related" functions
    - Bonus: Binary diffing might be reduced to sequence alignment (e.g., Dynamic Time Warping)
  - Modular decomposition
    - Partitioning of program functions into modules
    - Hierarchical structure with a unique (up to isomorphism) decomposition tree

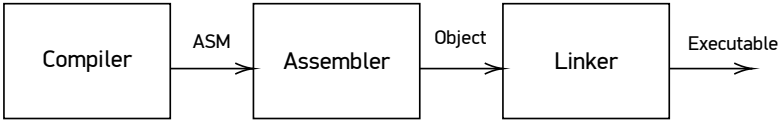
REcover: Approximate recovery of object file structure

https://github.com/huku-/recover

- Reverse engineering
  - Object files consist of semantically related functions (e.g., functions implementing a single feature)
  - Understand internals, apply analyses in a divide-and-conquer approach (e.g., binary diffing, malware classification etc.)
- Re-engineering
  - Map functionality of interest, debloat software, identify APIs and replace with own implementation
- Code re-use
  - Locate custom/proprietary cryptographic algorithms, extract and re-use relevant code
- Program understanding
  - Identity semantically related groups of functions, build and study software architecture graph

- Previous research on this problem (known as delinking or unlinking) is scarse:
  - Unlinker: an approach to identity original compilation units in stripped binaries (2016)
    - Main inspiration
    - Assumes the number of compile-units is known in advance
  - DeLink: Source File Information Recovery in Binaries (2024)
    - Uses limited-scope code patterns to detect cross-compile-unit function calls
- Recent work not citing my techniques and open-source tools:
  - BinStruct: Binary Structure Recovery Combining Static Analysis and Semantics (2025)
- From 2016 to 2024 no papers were published on this subject
  - Then I submitted to two famous journals
  - I was rejected
  - Suddenly, two papers appeared in a period of less than two years

- Typical compilation pipeline:



- Optimization passes can be applied at different stages:
  - At compile time
  - At link time (i.e., Link Time Optimization - LTO)
    - Hot-cold splitting of compile-units

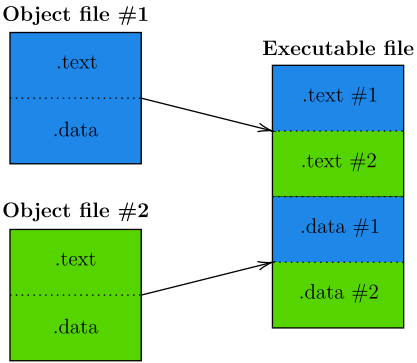


Figure: Object file section concatenation

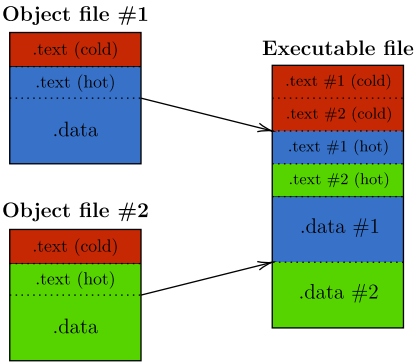
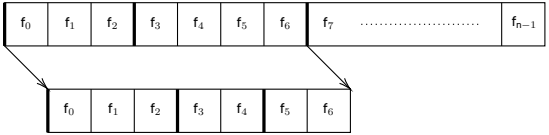


Figure: Hot-cold optimized object file section concatenation

- Can we recover the aforementioned object file structure?
- Two key questions:
  1. How many compile-units are there?
  2. What are their sizes?

- Model a program of N functions as a bit-vector of N bits
  - Set bits indicate split-points between object files.
  - In the figure f_0, f_3 and f_7 (i.e., bits 0, 3 and 7) are set



  - Idea attributed to Capelletti et al. (2016)
- Reformulated questions:
  1. How many bits are set?
  2. What is their position in the bit-vector?

- Construct the following graphs:
  - Function Call Graph (FCG)
  - Data Flow Graph (DFG)
  - Program and Data Graph (PDG) $\rightarrow$ FCG + DFG
    - Additional edges from functions to data elements and vice-versa
  - Augmented Function Call Graph (AFCG)
    - For each path $f_0 \rightarrow d_1 \rightarrow \ldots \rightarrow d_{n-1} \rightarrow f_n$, such that f_0, f_n are functions and d_i data elements, add an edge $f_0 \rightarrow f_n$.

      We add edges between functions which are connected via paths of data elements.

    - Sequence edges – Connect physically contiguous functions
    - Density edges – Connect functions called at the same call level (WIP)

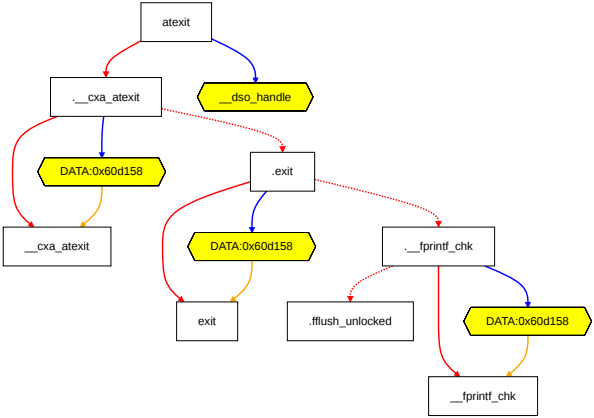
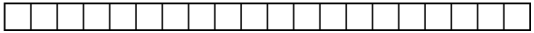


Figure: Partial PDG of sha256sum of coreutils-7.6-O3 from DeepBinDiff dataset

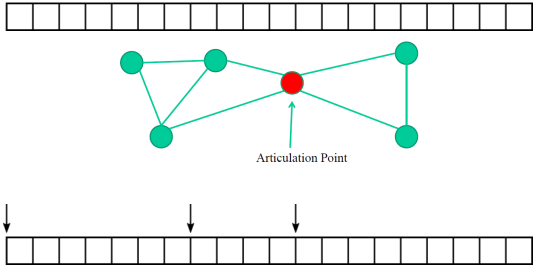
- Main compile-unit recovery algorithm:
  1. Build aforementioned graphs
  2. Run initial estimation algorithm (4 options):
     - apsnse
     - apspse
     - agglnse
     - agglpse
  3. Run optimization algorithm

Initial estimation algorithm #1: Articulation points



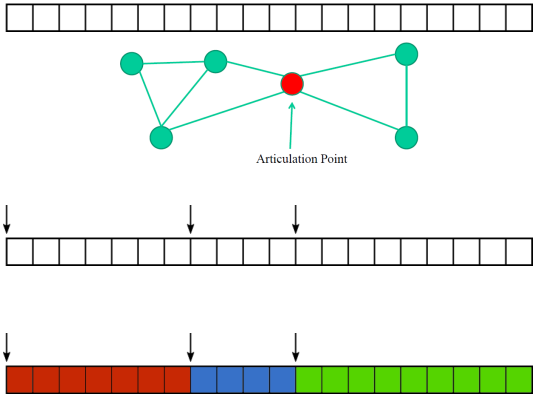
- Start with all functions in a single compile-unit

Initial estimation algorithm #1: Articulation points



- Start with all functions in a single compile-unit
- Run Hopcroft-Tarjan algorithm on the AFCG to determine articulation points
  - apspse - Articulation points - partial sequence edges
  - apsnse - Articulation points - no sequence edges

Initial estimation algorithm #1: Articulation points

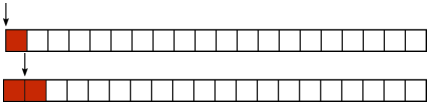


- Start with all functions in a single compile-unit
- Run Hopcroft-Tarjan algorithm on the AFCG to determine articulation points
  - apspse - Articulation points - partial sequence edges
  - apsnse - Articulation points - no sequence edges
- Use articulation points as split-points to partition the program into the set of compile-units

Initial estimation algorithm #2: Agglomeration

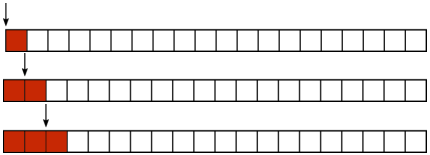
- Initially each function comprises a single compile-unit

Initial estimation algorithm #2: Agglomeration



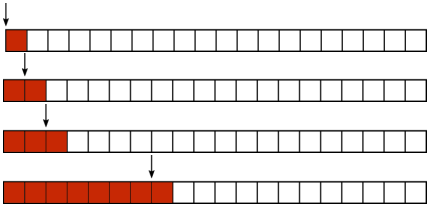
- Initially each function comprises a single compile-unit
- Functions are merged in a single compile-unit as long as the number of non-tree edges in the CUG decreases

Initial estimation algorithm #2: Agglomeration



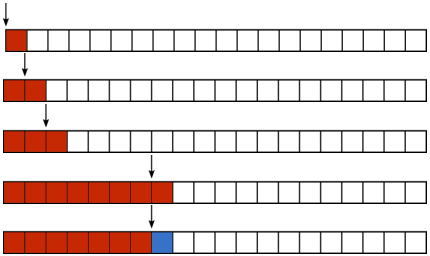
- Initially each function comprises a single compile-unit
- Functions are merged in a single compile-unit as long as the number of non-tree edges in the CUG decreases

Initial estimation algorithm #2: Agglomeration



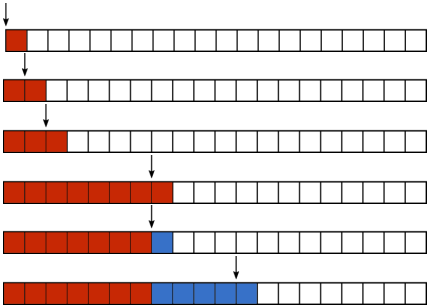
- Initially each function comprises a single compile-unit
- Functions are merged in a single compile-unit as long as the number of non-tree edges in the CUG decreases

Initial estimation algorithm #2: Agglomeration



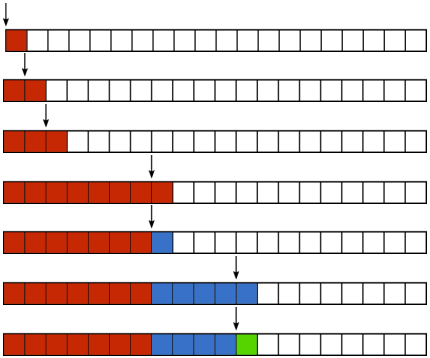
- Initially each function comprises a single compile-unit
- Functions are merged in a single compile-unit as long as the number of non-tree edges in the CUG decreases
- Otherwise current function stays in its compile-unit

Initial estimation algorithm #2: Agglomeration



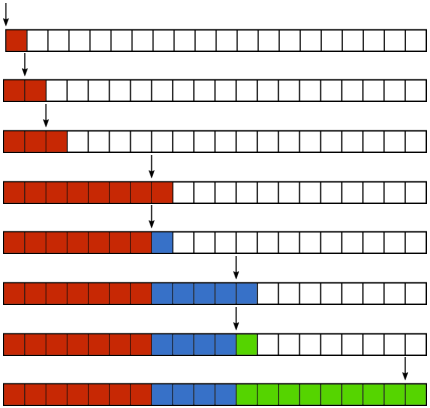
- Initially each function comprises a single compile-unit
- Functions are merged in a single compile-unit as long as the number of non-tree edges in the CUG decreases
- Otherwise current function stays in its compile-unit
- Process repeats until all functions are visited sequentially
  - agglpse - Agglomeration - partial sequence edges
  - agglnse - Agglomeration - no sequence edges

Initial estimation algorithm #2: Agglomeration



- Initially each function comprises a single compile-unit
- Functions are merged in a single compile-unit as long as the number of non-tree edges in the CUG decreases
- Otherwise current function stays in its compile-unit
- Process repeats until all functions are visited sequentially
  - agglpse - Agglomeration - partial sequence edges
  - agglnse - Agglomeration - no sequence edges

Initial estimation algorithm #2: Agglomeration



- Initially each function comprises a single compile-unit
- Functions are merged in a single compile-unit as long as the number of non-tree edges in the CUG decreases
- Otherwise current function stays in its compile-unit
- Process repeats until all functions are visited sequentially
  - agglpse - Agglomeration - partial sequence edges
  - agglnse - Agglomeration - no sequence edges

Initial estimation algorithm #2: Agglomeration

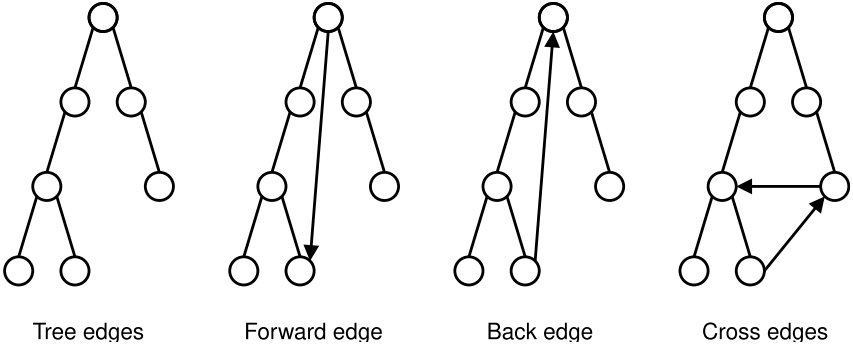
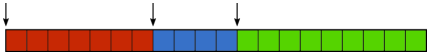


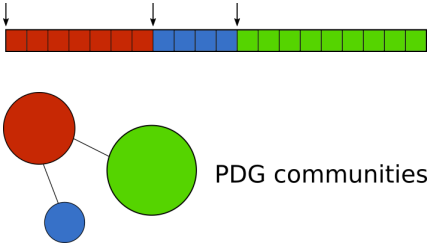
Figure: Non-tree edges

Optimization algorithm



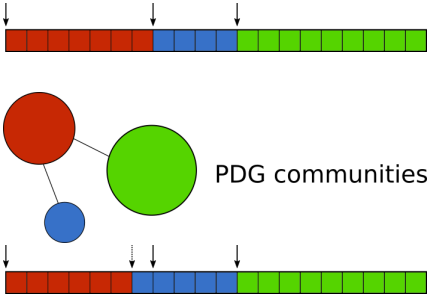
- Start with initial estimation

Optimization algorithm



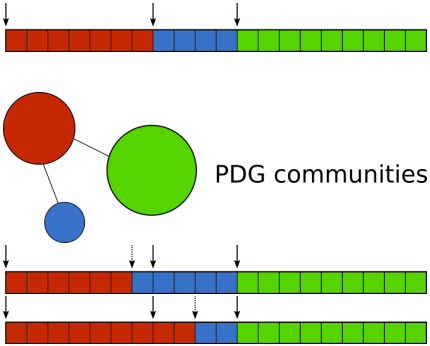
- Start with initial estimation
- Treat estimated compile-units as communities

Optimization algorithm



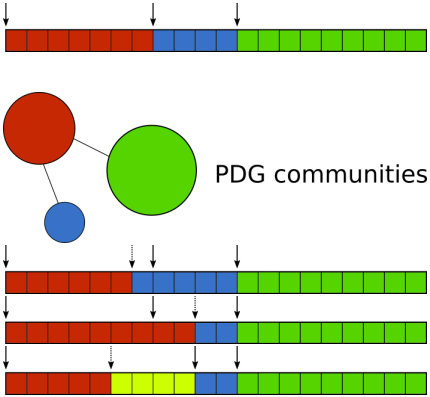
- Start with initial estimation
- Treat estimated compile-units as communities
- Visit compile-units in pairs and find the split with the highest Newman modularity. Start with red-blue pair.
  - Boundary may move to the left or right
  - An additional compile-unit might be created in between the current pair

Optimization algorithm



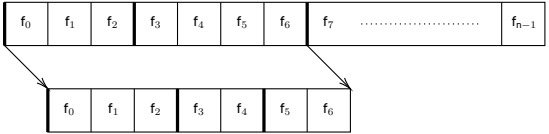
- Start with initial estimation
- Treat estimated compile-units as communities
- Visit compile-units in pairs and find the split with the highest Newman modularity. Start with red-blue pair.
  - Boundary may move to the left or right
  - An additional compile-unit might be created in between the current pair
- Repeat for all pairs (blue-green is next). Use work-list to keep optimizing as long as compile-unit boundaries change.

Optimization algorithm



- Start with initial estimation
- Treat estimated compile-units as communities
- Visit compile-units in pairs and find the split with the highest Newman modularity. Start with red-blue pair.
  - Boundary may move to the left or right
  - An additional compile-unit might be created in between the current pair
- Repeat for all pairs (blue-green is next). Use work-list to keep optimizing as long as compile-unit boundaries change.

- Optimization example:
  - Consider a pair of compile-units shown below f_0-f_2 and f_3-f_6
  - 7 functions, bit-vector of 7 bits



  - Generate all bit-vectors with 1, 2 or 3 bits set, such that bit for f_0 is always set (f_0 always marks the beginning of a compile-unit)
  - For each bit-vector form the corresponding PDG communities consisting of functions and data elements
  - Compute Newman modularity
  - Keep bit-vector with highest modularity

DEMO

- Had old FFmpeg binary on my hard disk, ran REcover, got ≈2100 compile-units
- FFmpeg binary had no debug symbols, but contained the `configure` command line arguments plus library versions etc.
- Built own binary using Android NDK 25.1.8937393
  - Ran REcover, got ≈1500 compile-units
- Re-built own binary using Android NDK r15c
  - Same NDK as original binary
  - Ran REcover, got ≈2000 compile-units → Success!
- Android NDK 25.1.8937393 → No hot-cold splitting of compile units

REcover

WIP

- REcover works quite well, but can be further improved
- We need increased edge density between functions that might belong to the same compile-unit
- Idea:
  - Collect callees of a function
  - Apply intra-procedural reaching-definitions
  - Find return values of functions that end up as input arguments to other functions
  - Might indicate functions of a single compile-unit acting on opaque data structures
- Experimented a bit with this
  - Results were interesting
  - But ended up with a more simple solution

REcover

WIP

- Introducing density edges:
  1. Collect callees of a function
  2. Sort by address
  3. Traverse sorted functions using a window of length W and add edges between them
  4. W a logarithmic function of the number of functions
- Over-approximation of what we would have achieved using the reaching-definitions approach
- Implemented but disabled for now:
  - https://github.com/huku-/recover/blob/master/src/recover/exporters/ida_pro.py#L333

Density edges

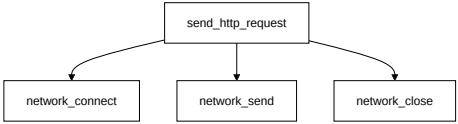


Figure: Three functions at the same call level

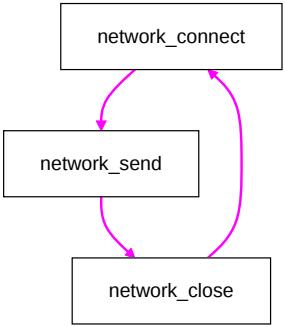


Figure: Add density edges

- Compile-unit recovery is hindered by various factors:
  - The ability of reverse engineering tools to identify functions and function boundaries
  - Presence of packed and/or obfuscated code
  - Hot-cold compile-unit splitting
    - Split parts are detected as separate compile-units
    - Not necessarily bad
  - LTO optimizers like BOLT and other exotic compiler command line options that intermix functions belonging to different compile-units
  - Function inlining and outlining
    - Introduce discrepancies between source code structure and binary code layout

Modular decomposition

Modular decomposition

About

- Modular decomposition (MD) is a graph decomposition algorithm
  - Clustering, partition, decomposition...?
  - Strictly speaking, decomposition indicates segmentation into sub-graphs
- Decomposes a graph into a set of modules

Modular decomposition

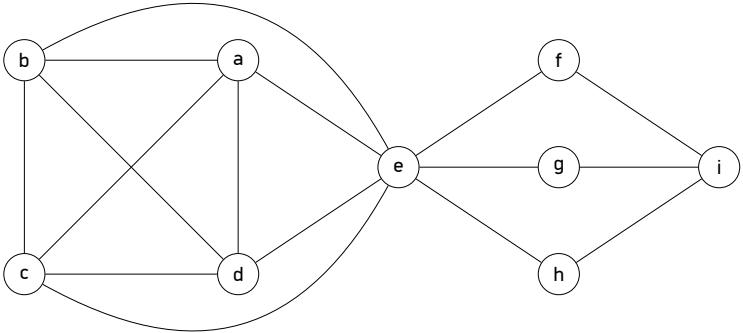
About

- Modules are a generalization of connected components
  - Connected components
    - All $v \in C$, C connected component, have the property that they are non-neighbors of all $u \notin C$
  - Modules
    - All $v \in M$, M a module, have the property that they are either neighbors or non-neighbors of all $u \notin M$
    - That is, all $v \in M$ have the same neighborhood outside M
    - Vertices in M are referred to as indistinguishable

Modular decomposition

Example

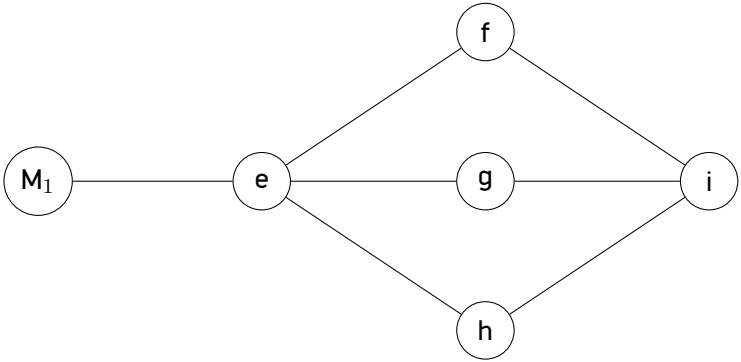
A graph consisting of 9 nodes and 16 edges



Modular decomposition

Example

Replace $\{a, b, c, d\}$ with a series module M_1



Modular decomposition

Example

Replace $\{f, g, h\}$ with a parallel module M_2

Modular decomposition

About

- Modules can be of the following types:
  - SERIES $\rightarrow$ Module is a clique (all vertices connected to each other)
    - $\{a, b, c, d\}$ in previous example
  - PARALLEL $\rightarrow$ Module is an independent set (all vertices are isolated)
    - $\{f, g, h\}$ in previous example
  - PRIME $\rightarrow$ Module does not belong to any of the above categories

Modular decomposition

About

- Modules are nested
  - Detected modules form a quotient graph
    - Whole modules replaced by single vertices
    - Edges connect modules if vertices of these modules are connected
  - Eventually a tree is formed $\rightarrow$ MD-tree
  - MD-tree is unique up to isomorphism!

Modular decomposition

Algorithm

- Algorithms behind modular decomposition are non-trivial
- Research on the topic back to the 1960s
- Algorithms evolved from $O(n^4)$ to $O(n)$
- State of the art:
  - Efficient Implementation of Modular Graph Decomposition (2023)
- Implemented it in NetworkX and submitted pull-request
  - `https://github.com/networkx/networkx/pull/7957`

Modular decomposition

Effect of compiler optimizations - coreutils-9.1 / shred

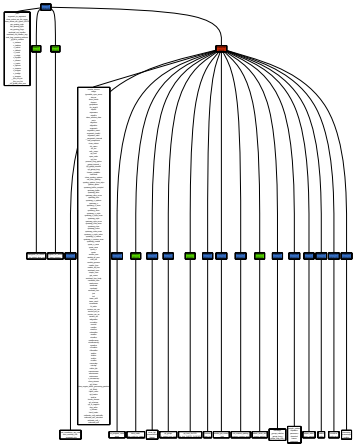


Figure: AMD64 / gcc-11.2.0 / O0

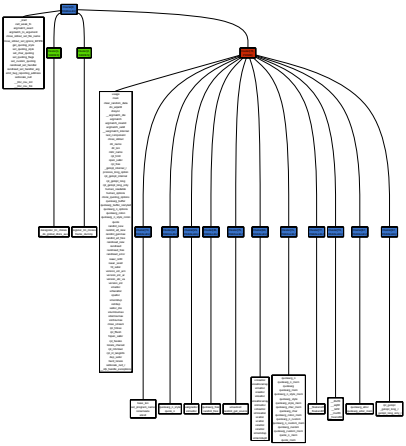


Figure: AArch64 / clang-11.0 / O3

Modular decomposition

Effect of compiler optimizations - coreutils-8.30 vs. coreutils-9.1 / shred

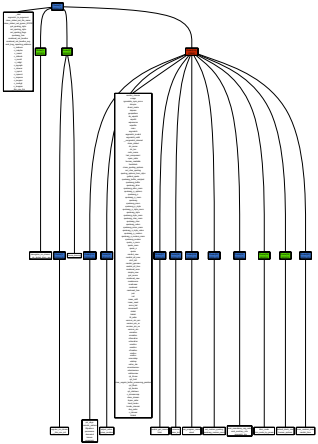


Figure: 8.30 / AMD64 / gcc-7.4.0 / O0

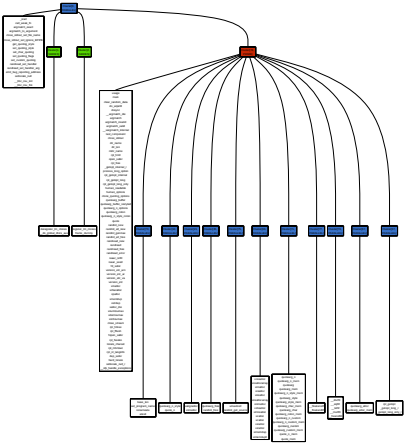


Figure: 9.1 / AArch64 / clang-11.0 / O3

Modular decomposition

Is it worth it?

- Advantages:
  - MD-tree unique up to isomorphism
  - Resilient to certain compiler optimizations
  - Parallel Least Common Ancestor (LCA) in the MD-tree indicates complete independence
  - Series LCA full dependence
- Disadvantages:
  - Prime modules give no insight on the internal relationship constituent of functions
  - Same for a prime LCA in the MD-tree
  - Small number of large prime modules
  - Large number of small parallel and series modules

Modular decomposition

Is it worth it?

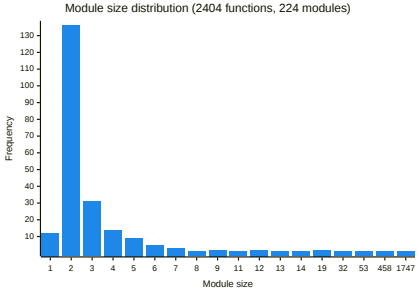


Figure: Histogram of module sizes of binutils-2.40_clang-13.0_arm_64_Os_objdump

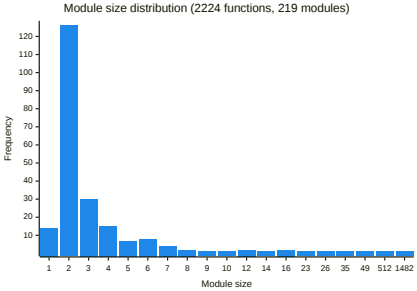


Figure: Histogram of module sizes of binutils-2.40_clang-13.0_arm_64_O3_objdump

Modular decomposition

- Turns out modular decomposition is just a specific instance of a wider combinatorial decomposition framework for various structures (e.g., graphs, matroids, boolean functions etc.)

Can. J. Math., Vol. XXXII, No. 3, 1980, pp. 734–765

A COMBINATORIAL DECOMPOSITION THEORY

WILLIAM H. CUNNINGHAM AND JACK EDMONDS

1. Introduction. Given a finite undirected graph G and $A \subseteq E(G)$, $G(A)$ denotes the subgraph of G having edge-set A and having no isolated vertices. For a partition $\{E_1, E_2\}$ of $E(G)$, $W(G; E_1)$ denotes the set $V(G(E_1)) \cap V(G(E_2))$. We say that G is *non-separable* if it is connected and for every proper, non-empty subset A of $E(G)$, we have $|W(G; A)| \geqq 2$. A *split* of a non-separable graph G is a partition $\{E_1, E_2\}$ of $E(G)$ such that

$$|E_1| \geqq 2 \leqq |E_2| \text{ and } |W(G; E_1)| = 2.$$

DEMO

Conclusions

Conclusions

- We looked at two binary segmentation techniques; REcover and modular decomposition
- Both have advantages and disadvantages; REcover appears more promising
- REcover has been submitted to the IDA Pro plug-in contest (fingers crossed), while support for Binary Ninja is coming the following days, with contributions by Athanasios Kostopoulos!
- Have used these techniques for divide-and-conquer binary diffing
  - Details will be presented in my PhD thesis
  - Expected around May 2026

Questions

Questions? Kthanxbai!